

Ανάλυση Αποφάσεων και Βελτιστοποίηση

Εργασία 2

N. Τσάντας

Αλέξανδρος Τσαγκανός

18 Ιανουαρίου 2026

1 Άσκηση 1: Βελτιστοποίηση Καυσίμων Αεροσκά- φους (Fuel Tankering)

1.1 Περιγραφή Προβλήματος

Σκοπός της άσκησης είναι η βελτιστοποίηση του προγράμματος ανεφοδιασμού ενός αεροσκά-
φους που εκτελεί το δρομολόγιο **Αθήνα (ATH) → Λονδίνο (LON) → Παρίσι (PAR) → Ρώμη (ROM) → Αθήνα (ATH)**. Το πρόβλημα αφορά την εξισορρόπηση δύο παραγόντων:

1. Της αγοράς φθηνού καυσίμου σε συγκεκριμένα αεροδρόμια (Tankering).
2. Του κόστους μεταφοράς του επιπλέον βάρους καυσίμου (Fuel Penalty).

Εξετάζονται δύο σενάρια: Μονός Κύκλος (Scenario A) και Επαναλαμβανόμενοι Κύκλοι (Scenario B).

1.2 Μαθηματικό Μοντέλο

Μεταβλητές Απόφασης:

- x_i : Ποσότητα καυσίμου που αγοράζεται στο αεροδρόμιο i (σε 1000 Λίτρα).
- d_i : Καύσιμο κατά την αναχώρηση από το αεροδρόμιο i .
- a_i : Καύσιμο κατά την άφιξη στο αεροδρόμιο i .

Αντικειμενική Συνάρτηση: Ελαχιστοποίηση συνολικού κόστους αγοράς:

$$\min Z = \sum_{i \in \{ATH, LON, PAR, ROM\}} P_i \cdot x_i$$

Όπου P_i η τιμή καυσίμου στο εκάστοτε αεροδρόμιο.

Περιορισμοί:

1. **Ισοζύγιο Μάζας:** $d_i = a_i + x_i$ (Αναχώρηση = Άφιξη + Αγορά).
2. **Κατανάλωση:** $a_{i+1} = d_i - (BaseBurn_i + \alpha \cdot (d_i - MinFuel_i))$.
3. **Όρια Ασφαλείας:** $MinFuel_i \leq d_i \leq MaxFuel_i$.
4. **Περιορισμός Σεναρίου B:** $a_{ATH_Start} = a_{ATH_End}$ (Το καύσιμο που μένει στο τέλος είναι διαθέσιμο για την αρχή του επόμενου κύκλου).

1.3 Αποτελέσματα και Ανάλυση

Σενάριο Α (Μονός Κύκλος): Συνολικό Κόστος: **38,121 €**. Το αεροσκάφος αγοράζει μεγάλη ποσότητα στην Αθήνα (φθηνότερη τιμή 1.15€) και προσγειώνεται πίσω στην Αθήνα με 6,000 λίτρα υπόλοιπο.

Σενάριο Β (Επαναλαμβανόμενοι Κύκλοι): Συνολικό Κόστος: **31,221 €**. Λόγω της περιοδικότητας, τα 6,000 λίτρα που περισσεύουν χρησιμοποιούνται στην επόμενη πτήση. Έτσι, η αγορά στην Αθήνα μειώνεται από 27,158 σε 21,158 λίτρα. **Εξοικονόμηση:** 6,900 € ανά κύκλο.

Table 1: Βέλτιστο Σχέδιο Ανεφοδιασμού (Σενάριο Β)

Αεροδρόμιο	Τιμή (€)	Αγορά (L)	Αναχώρηση (L)	Άφιξη (L)
Αθήνα	1.15	21,158	27,158	6,000
Λονδίνο	1.60	0	21,833	21,833
Παρίσι	1.70	6,263	22,500	16,237
Ρώμη	1.65	0	16,500	16,500

Συμπέρασμα: Η βέλτιστη στρατηγική είναι το "Tankering" από την Αθήνα, αποφεύγοντας πλήρως την αγορά καυσίμων σε Λονδίνο και Ρώμη, και συμπληρώνοντας μόνο το απαραίτητο στο Παρίσι.

Άσκηση 2: Βελτιστοποίηση Παραγωγής Διανομής (Wallasidis Juice Company)

Περιγραφή Προβλήματος

Η εταιρεία "Wallasidis Juice Company" επιδιώκει την ελαχιστοποίηση του συνολικού κόστους (μεταφοράς και επεξεργασίας) για την παραγωγή τριών προϊόντων (Εμφιαλωμένος Χυμός, Κατεψυγμένος Συμπυκνωμένος, Ζελέ) από σταφύλια τριών αμπελώνων (Α, Β, Γ) που μεταφέρονται σε τέσσερα εργοστάσια (Κρήτη, Ηλεία, Αττική, Μακεδονία).

Μαθηματικό Μοντέλο

Σύνολα και Δείκτες:

- $I = \{A, B, \Gamma\}$: Αμπελώνες (Προέλευση).
- $J = \{Kr, Il, At, Ma\}$: Εργοστάσια (Προορισμός/Επεξεργασία).

- $K = \{Bot, Fro, Jel\}$: Τελικά Προϊόντα.

Μεταβλητές Απόφασης:

- $X_{ij} \geq 0$: Ποσότητα μη επεξεργασμένου χυμού που μεταφέρεται από τον αμπελώνα i στο εργοστάσιο j (τόνοι).
- $Y_{jk} \geq 0$: Ποσότητα τελικού προϊόντος k που παράγεται στο εργοστάσιο j (τόνοι).

Παράμετροι:

- $Supply_i$: Διαθέσιμη ποσότητα στον αμπελώνα i .
- $Capacity_j$: Δυναμικότητα επεξεργασίας (εισερχόμενου χυμού) στο εργοστάσιο j .
- $Demand_k$: Ζήτηση για το προϊόν k .
- $TransCost_{ij}$: Κόστος μεταφοράς από i σε j (€/τόνο).
- $ProcCost_{jk}$: Κόστος επεξεργασίας προϊόντος k στο εργοστάσιο j (€/τόνο).
- $Rate_k$: Συντελεστής μετατροπής (τόνοι πρώτης ύλης ανά τόνο προϊόντος).
 - $Rate_{Bot} = 1.0$
 - $Rate_{Fro} = 2.0$
 - $Rate_{Jel} = 1.5$

Αντικειμενική Συνάρτηση (Ελαχιστοποίηση Κόστους):

$$\min Z = \sum_{i \in I} \sum_{j \in J} TransCost_{ij} X_{ij} + \sum_{j \in J} \sum_{k \in K} ProcCost_{jk} Y_{jk}$$

Περιορισμοί:

1. Περιορισμοί Προσφοράς Αμπελώνων:

$$\sum_{j \in J} X_{ij} \leq Supply_i, \quad \forall i \in I$$

2. Περιορισμοί Δυναμικότητας Εργοστασίων (Εισροές):

$$\sum_{i \in I} X_{ij} \leq Capacity_j, \quad \forall j \in J$$

3. Ισοζύγιο Μάζας στα Εργοστάσια (Εισροή = Κατανάλωση για Παραγωγή):

$$\sum_{i \in I} X_{ij} = \sum_{k \in K} (Rate_k \cdot Y_{jk}), \quad \forall j \in J$$

4. Περιορισμοί Ζήτησης Προϊόντων:

$$\sum_{j \in J} Y_{jk} = Demand_k, \quad \forall k \in K$$

Αποτελέσματα και Ανάλυση Ευαισθησίας

Βέλτιστη Λύση: Το ελάχιστο συνολικό κόστος ανέρχεται στα **10.606.000 €**.

Table 2: Βέλτιστο Σχέδιο Μεταφορών (Τόννοι)

Από/Προς	Κρήτη	Ηλεία	Αττική	Μακεδονία
Αμπελώνας Α	0	0	0	1400
Αμπελώνας Β	0	1100	0	0
Αμπελώνας Γ	150	0	1400	0

Table 3: Βέλτιστο Σχέδιο Παραγωγής (Τόννοι)

Εργοστάσιο	Εμφιαλωμένος	Κατεψυγμένος	Ζελέ
Κρήτη	0	75	0
Ηλεία	0	25	700
Αττική	0	700	0
Μακεδονία	1200	100	0

Ανάλυση Ευαισθησίας (Shadow Prices & Reduced Costs):

1. Δεξιά Μέλη (Right-Hand Side Sensitivity):

- **Δυναμικότητα Αττικής (Shadow Price = -195):** Η οριακή τιμή για τη χωρητικότητα της Αττικής είναι -195. Αυτό σημαίνει ότι για κάθε επιπλέον τόνο δυναμικότητας που αποκτά το εργοστάσιο της Αττικής, το συνολικό κόστος μειώνεται κατά 195€. Αυτό συμβαίνει διότι η Αττική είναι πολύ αποδοτική για την παραγωγή Κατεψυγμένου χυμού (σε συνδυασμό με τη μεταφορά από τον Αμπελώνα Γ), και η τρέχουσα χωρητικότητα περιορίζει την εκμετάλλευση αυτού του πλεονεκτήματος.
- **Προσφορά Αμπελώνα Α (Shadow Price = -70):** Η αύξηση της διαθέσιμης ποσότητας στον Αμπελώνα Α κατά 1 τόνο θα μείωνε το κόστος κατά 70€. Ο Αμπελώνας Α τροφοδοτεί τη Μακεδονία με χαμηλό κόστος μεταφοράς (750€), και η πλήρης αξιοποίησή του δείχνει ότι είναι μια πολύτιμη πηγή πρώτης ύλης.

2. Συντελεστές Αντικειμενικής Συνάρτησης (Objective Coefficient Sensitivity):

- **Μεταφορά Α → Κρήτη (Reduced Cost = 20):** Το τρέχον κόστος είναι 850€. Το Reduced Cost είναι 20, που σημαίνει ότι το κόστος μεταφοράς πρέπει να μειωθεί τουλάχιστον κατά 20€ (δηλαδή να γίνει < 830) για να συμφέρει η χρήση αυτής της διαδρομής στη βέλτιστη λύση.

- **Παραγωγή Εμφιαλωμένου στην Κρήτη (Reduced Cost = 100):** Το κόστος επεξεργασίας είναι 2100€. Θα πρέπει να μειωθεί κατά 100€ (στα 2000€) για να ενταχθεί στο πλάνο παραγωγής, καθώς αυτή τη στιγμή άλλες επιλογές (π.χ. Μακεδονία) είναι πιο οικονομικές.

Άσκηση 3: Βελτιστοποίηση Διατροφής (Πρόβλημα Δίαιτας)

Περιγραφή Προβλήματος

Η Εύα επιθυμεί να μεγιστοποιήσει τη "γευστικότητα" του γεύματός της, επιλέγοντας τον κατάλληλο συνδυασμό δύο τύπων γευμάτων (Α και Β), τηρώντας παράλληλα αυστηρούς διατροφικούς περιορισμούς (θερμίδες, λιπαρά, ελάχιστο βάρος τροφής).

α) Μαθηματικό Μοντέλο

Μεταβλητές Απόφασης:

- x_1 : Αριθμός μερίδων γεύματος τύπου Α.
- x_2 : Αριθμός μερίδων γεύματος τύπου Β.

Παράμετροι (ανά μερίδα):

- **Γεύμα Α:** Βάρος 37g, Θερμίδες 120, Λιπαρά 5g.
- **Γεύμα Β:** Βάρος 65g, Θερμίδες 160, Λιπαρά 10g.
- **Δείκτης Γευστικότητας:**
 - Γεύμα Α: $85 \text{ μονάδες/g} \times 37\text{g} = 3145 \text{ μονάδες/μερίδα}$.
 - Γεύμα Β: $95 \text{ μονάδες/g} \times 65\text{g} = 6175 \text{ μονάδες/μερίδα}$.

Αντικειμενική Συνάρτηση (Μεγιστοποίηση Γευστικότητας):

$$\max Z = 3145x_1 + 6175x_2$$

Περιορισμοί:

1. **Θερμίδες (Max 450):**

$$120x_1 + 160x_2 \leq 450$$

2. Λιπαρά (Max 25g):

$$5x_1 + 10x_2 \leq 25$$

3. Ελάχιστο Βάρος Τροφής (Min 120g):

$$37x_1 + 65x_2 \geq 120$$

4. Μη αρνητικότητα:

$$x_1, x_2 \geq 0$$

b) Γραφική Μέθοδος και Εφικτή Περιοχή

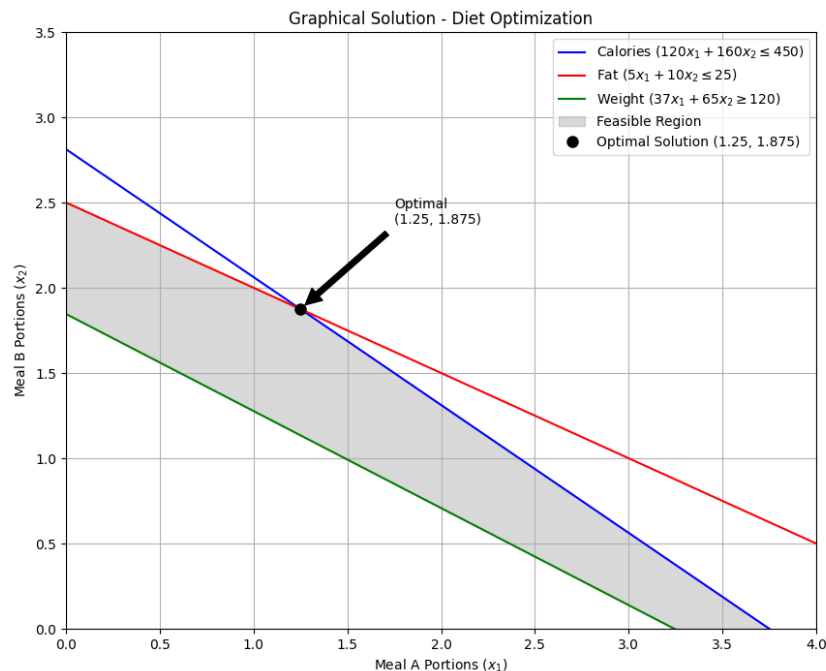


Figure 1: Γραφική απεικόνιση της εφικτής περιοχής και της βέλτιστης λύσης.

Για να βρούμε τη βέλτιστη λύση γραφικά, σχεδιάζουμε τις ευθείες που αντιστοιχούν στους περιορισμούς στο επίπεδο (x_1, x_2) .

- Η ευθεία των θερμίδων ($120x_1 + 160x_2 = 450$) τέμνει τους άξονες στα σημεία $(3.75, 0)$ και $(0, 2.81)$.
- Η ευθεία των λιπαρών ($5x_1 + 10x_2 = 25$) τέμνει τους άξονες στα σημεία $(5, 0)$ και $(0, 2.5)$.
- Η ευθεία του βάρους ($37x_1 + 65x_2 = 120$) ορίζει το κάτω όριο της περιοχής.

Η **Εφικτή Περιοχή** είναι το πολύγωνο που περικλείεται από τις τομές αυτών των ευθειών. Η βέλτιστη λύση βρίσκεται συνήθως σε μία από τις κορυφές της εφικτής περιοχής. Εξετάζοντας την κλίση της αντικειμενικής συνάρτησης, το βέλτιστο σημείο εντοπίζεται στην τομή των περιορισμών Θερμίδων και Λιπαρών.

Λύνοντας το σύστημα των εξισώσεων:

$$\begin{cases} 120x_1 + 160x_2 = 450 \\ 5x_1 + 10x_2 = 25 \end{cases}$$

Πολλαπλασιάζοντας τη δεύτερη εξίσωση με -16: $-80x_1 - 160x_2 = -400$. Προσθέτοντας στην πρώτη: $40x_1 = 50 \Rightarrow x_1 = 1.25$. Αντικαθιστώντας: $5(1.25) + 10x_2 = 25 \Rightarrow 6.25 + 10x_2 = 25 \Rightarrow 10x_2 = 18.75 \Rightarrow x_2 = 1.875$.

Άριστη Λύση: $x_1 = 1.25$ μερίδες A, $x_2 = 1.875$ μερίδες B. **Μέγιστη Γευστικότητα:** $Z = 3145(1.25) + 6175(1.875) = 15509.375$.

Συνολικά Στοιχεία Λύσης:

- Συνολικό Βάρος: $37(1.25) + 65(1.875) = 168.125$ γραμμάρια.
- Συνολικές Θερμίδες: $120(1.25) + 160(1.875) = 450$ (Όριο).
- Συνολικά Λιπαρά: $5(1.25) + 10(1.875) = 25$ γραμμάρια (Όριο).

c) Επίλυση με Python (Pulp) Παράρτημα C

Αποτελέσματα Κώδικα:

- Meal A (x_1): 1.25
- Meal B (x_2): 1.875
- Max Tastiness (Z): 15509.375
- Shadow Price (Calories): 1.4375
- Shadow Price (Fat): 594.5

d) Ανάλυση Σεναρίου: Αλλαγή Γευστικότητας Γεύματος A

Αν ο δείκτης γευστικότητας του Γεύματος A γίνει 83 μονάδες/g, ο νέος συντελεστής στην αντικειμενική συνάρτηση γίνεται: $c'_1 = 83 \times 37 = 3071$. Η νέα αντικειμενική συνάρτηση είναι: $Z' = 3071x_1 + 6175x_2$.

Επιλύοντας ξανά το μοντέλο με τη νέα τιμή, η βέλτιστη λύση αλλάζει. Η κλίση της αντικειμενικής συνάρτησης γίνεται πιο "επίπεδη", καθιστώντας πιο συμφέρουσα την αποκλειστική κατανάλωση του Γεύματος Β (μέχρι να εξαντληθεί το όριο των λιπαρών).

Νέα Βέλτιστη Λύση:

- $x_1 = 0$ (Καμία μερίδα Α)
- $x_2 = 2.5$ (Μερίδες Β)
- Νέα Τιμή Z : 15437.5

Σε αυτή την περίπτωση, περιοριστικός παράγοντας είναι μόνο τα λιπαρά ($2.5 \times 10 = 25g$), ενώ οι θερμίδες είναι $2.5 \times 160 = 400 < 450$.

ε) Ανάλυση Ευαισθησίας: Μείωση Ορίου Θερμίδων

Αν το ανώτερο όριο θερμίδων μειωθεί κατά 10 μονάδες (από 450 σε 440), μπορούμε να υπολογίσουμε τη μεταβολή στη γευστικότητα χρησιμοποιώντας την **Οριακή Τιμή (Shadow Price)** του περιορισμού των θερμίδων, χωρίς να επιλύσουμε ξανά το μοντέλο.

Από την επίλυση στο ερώτημα (ε), η Shadow Price για τις θερμίδες είναι **1.4375**. Αυτό σημαίνει ότι για κάθε μονάδα θερμίδας που χάνουμε, η συνολική γευστικότητα μειώνεται κατά 1.4375 μονάδες.

$$\Delta Z = \text{Shadow Price} \times \Delta(\text{RHS})$$

$$\Delta Z = 1.4375 \times (-10) = -14.375$$

Επομένως, η νέα μέγιστη γευστικότητα θα είναι:

$$Z_{\text{new}} = 15509.375 - 14.375 = 15495$$

Άσκηση 4: Ανθρωπιστική Βοήθεια στην Κούβα (Helping Hand)

Περιγραφή Προβλήματος και Δεδομένα

Ο οργανισμός "Helping Hand" επιδιώκει τη βέλτιστη κατανομή τριών τύπων πακέτων βοήθειας (Basic, Advanced, Supreme) στην Κούβα. Το πρόβλημα διατυπώνεται ως πρόβλημα

****Προγραμματισμού Στόχων (Goal Programming)****, καθώς υπάρχουν πολλαπλοί, αντικρουόμενοι στόχοι (κάλυψη πληθυσμού, αριθμός πακέτων, προϋπολογισμός) με διαφορετικά επίπεδα προτεραιότητας.

Χαρακτηριστικά Πακέτων:

Τύπος	Κόστος (\$)	Βάρος (lbs)	Άτομα που βοηθούνται
Basic (x_1)	300	120	30
Advanced (x_2)	350	180	35
Supreme (x_3)	720 + Γιατρός	220	54

Υπολογισμός Κόστους Supreme: Για κάθε 100 πακέτα Supreme απαιτείται 1 γιατρός με κόστος \$33,000.

$$\text{Κόστος Γιατρού ανά πακέτο} = \frac{33,000}{100} = 330\$$$

$$\text{Συνολικό Κόστος Supreme} = 720 + 330 = 1,050\$$$

Μαθηματικό Μοντέλο

Μεταβλητές Απόφασης: $x_1, x_2, x_3 \in \mathbb{Z}_{\geq 0}$ (αριθμός πακέτων).

Σκληροί Περιορισμοί (Hard Constraints): Οι φυσικοί περιορισμοί που δεν μπορούν να παραβιαστούν:

1. **Μέγιστος Αριθμός Πακέτων:** $x_1 + x_2 + x_3 \leq 40,000$
2. **Μέγιστο Βάρος:** $120x_1 + 180x_2 + 220x_3 \leq 6,000,000$

Περιορισμοί Στόχων (Soft Constraints): Εισάγουμε μεταβλητές απόκλισης d_i^- (υστέρηση) και d_i^+ (υπέρβαση).

1. **Στόχος Πληθυσμού (20% του 11εκ = 2,200,000):**

$$30x_1 + 35x_2 + 54x_3 + d_1^- - d_1^+ = 2,200,000$$

2. **Στόχος Supreme Πακέτων (Min 3,000):**

$$x_3 + d_2^- - d_2^+ = 3,000$$

3. **Στόχος Προϋπολογισμού (Max \$20εκ):**

$$300x_1 + 350x_2 + 1050x_3 + d_3^- - d_3^+ = 20,000,000$$

Ανάλυση Ερωτημάτων

a. Αρχική Κατανομή (Weighted Goal Programming)

Ο κ. Baker ορίζει συγκεκριμένες ποινές για τις αποκλίσεις. Πρέπει να μετατρέψουμε αυτές τις ποινές σε συντελεστές βαρύτητας (w_i) για την αντικειμενική συνάρτηση ελαχιστοποίησης: $\min Z = w_1 d_1^- + w_2 d_2^- + w_3 d_3^+$.

Υπολογισμός Βαρών:

- **Πληθυσμός (w_1):** 7 πόντοι για κάθε 100,000 άτομα υστέρησης.

$$w_1 = \frac{7}{100,000} = 0.00007$$

- **Supreme Πακέτα (w_2):** 1 πόντος για κάθε 1,000 πακέτα υστέρησης.

$$w_2 = \frac{1}{1,000} = 0.001$$

- **Προϋπολογισμός (w_3):** 1 πόντος για κάθε \$1,000,000 υπέρβασης.

$$w_3 = \frac{1}{1,000,000} = 0.000001$$

Αποτέλεσμα Επίλυσης(Παράρτημα Κώδικα Άσκησης 4):

- x_1 (Basic): **28,000**
- x_2 (Advanced): **0**
- x_3 (Supreme): **12,000**
- Συνολικό Κόστος: **\$21,000,000** (Υπέρβαση \$1εκ)
- Πληθυσμός που καλύπτεται: **1,488,000** (Υστέρηση 712,000)

Ερμηνεία: Παρατηρούμε ότι το βάρος της ποινής για τον πληθυσμό ($7 \cdot 10^{-5}$) είναι σημαντικά μεγαλύτερο από το βάρος του κόστους ($1 \cdot 10^{-6}$). Το μοντέλο επιλέγει να παραβιάσει τον στόχο του προϋπολογισμού για να μεγιστοποιήσει τον πληθυσμό. Η λύση καθορίζεται από τους σκληρούς περιορισμούς:

$$\text{Βάρος} = 28000(120) + 12000(220) = 3,360,000 + 2,640,000 = 6,000,000 \text{ lbs}$$

Το αεροσκάφος γεμίζει πλήρως σε βάρος. Τα πακέτα Supreme επιλέγονται γιατί εξυπηρετούν περισσότερα άτομα ανά πακέτο, μέχρι το σημείο που το βάρος τους γίνεται απαγορευτικό, οπότε συμπληρώνεται το φορτίο με ελαφρύτερα Basic πακέτα.

b. Ανάλυση Ευαισθησίας στα Βάρη

Ο κ. Baker αυξάνει την ποινή για τον πληθυσμό: 10 πόντοι για κάθε 0.5% του πληθυσμού (δηλαδή για κάθε 55,000 άτομα).

$$w'_1 = \frac{10}{55,000} \approx 0.000182$$

Το νέο βάρος είναι σχεδόν 2.5 φορές μεγαλύτερο από το προηγούμενο.

Αποτέλεσμα: Η λύση παραμένει **αμετάβλητη** (28,000 Basic, 12,000 Supreme). **Εξήγηση:** Στο ερώτημα (a), το μοντέλο είχε ήδη εξαντλήσει τους φυσικούς πόρους (Μέγιστο Βάρος και Μέγιστος Αριθμός Πακέτων) για να μεγιστοποιήσει τον πληθυσμό, αγνοώντας το κόστος. Η περαιτέρω αύξηση της σημαντικότητας του πληθυσμού δεν μπορεί να παράγει περισσότερα πακέτα, καθώς οι φυσικοί περιορισμοί είναι δεσμευτικοί (binding constraints). Το σχέδιο είναι ανελαστικό σε αυτή την αλλαγή.

c. Αλλαγή Κόστους (Περισσότεροι Γιατροί)

Απαιτείται 1 γιατρός ανά 75 πακέτα Supreme.

$$\text{Νέο Κόστος Γιατρού} = \frac{33,000}{75} = 440\$$$

$$\text{Νέο Κόστος Supreme} = 720 + 440 = 1,160\$$$

Αποτέλεσμα: Η λύση παραμένει **αμετάβλητη** ως προς τις ποσότητες (28,000/0/12,000), αλλά το συνολικό κόστος αυξάνεται στα **\$22,320,000**. **Εξήγηση:** Παρόλο που το κόστος των Supreme αυξήθηκε, η "αξία" τους σε άτομα (54 άτομα/πακέτο) σε συνδυασμό με την υψηλή ποινή για τη μη κάλυψη πληθυσμού, εξακολουθεί να τα καθιστά ελκυστικά. Η ποινή υπέρβασης προϋπολογισμού παραμένει πολύ χαμηλή για να επηρεάσει τη λύση.

d. Αυστηρός Περιορισμός Προϋπολογισμού

Ο προϋπολογισμός γίνεται σκληρός περιορισμός: $\sum Cost \leq 20,000,000$.

Αποτέλεσμα Επίλυσης(Παράρτημα Κώδικα Άσκησης 5):

- x_1 (Basic): **27,000**
- x_2 (Advanced): **2,500**
- x_3 (Supreme): **10,500**
- Κόστος: **\$20,000,000** (Ακριβώς στο όριο)

- Βάρος: **6,000,000 lbs** (Ακριβώς στο όριο)
- Πλήθος: **40,000** (Ακριβώς στο όριο)

Εξήγηση: Για να μειωθεί το κόστος από τα \$21εκ στα \$20εκ, το μοντέλο αναγκάζεται να μειώσει τα ακριβά Supreme πακέτα (από 12,000 σε 10,500). Για να διατηρηθεί το συνολικό πλήθος στα 40,000 και το βάρος στα 6εκ lbs (ώστε να μεγιστοποιηθεί ο πληθυσμός υπό τους νέους όρους), εισάγονται τα Advanced πακέτα ως ενδιάμεση λύση βάρους/κόστους. Η λύση αυτή αποτελεί το μοναδικό σημείο τομής όλων των περιορισμών (Κόστος, Βάρος, Πλήθος).

e. Preemptive Goal Programming (Ιεραρχική Προσέγγιση)

Εδώ οι στόχοι ιεραρχούνται απόλυτα:

1. **Προτεραιότητα P_1 :** Προϋπολογισμός (Να μην ξεπεραστεί).
2. **Προτεραιότητα P_2 :** Κάλυψη Πληθυσμού.
3. **Προτεραιότητα P_3 :** Supreme Πακέτα.

Αποτέλεσμα: Ίδιο με το ερώτημα (d) (27, 000/2, 500/10, 500). **Εξήγηση:** Στην ιεραρχική μέθοδο, ο αλγόριθμος πρώτα ικανοποιεί τον στόχο P_1 . Αυτό ουσιαστικά μετατρέπει τον στόχο του προϋπολογισμού σε σκληρό περιορισμό (όπως στο d). Στη συνέχεια, προσπαθεί να βελτιστοποιήσει τον πληθυσμό (P_2) μέσα στον εφικτό χώρο που άφησε ο P_1 . Καθώς η λύση του (d) ήταν ήδη η βέλτιστη δυνατή χρήση των πόρων για τον πληθυσμό δεδομένου του ορίου των \$20εκ, η ιεραρχική μέθοδος καταλήγει στο ίδιο ακριβώς σημείο.

Άσκηση 5: Σχολική Επιτροπή Oakdale (School Busing)

Περιγραφή Προβλήματος

Η Σχολική Επιτροπή της κομητείας Oakdale πρέπει να καταρτίσει ένα σχέδιο μεταφοράς μαθητών (busing plan) για να επιτύχει φυλετική ισορροπία στα 4 λύκεια της περιοχής (North, South, East, West). Το πρόβλημα διατυπώνεται ως ****Preemptive Goal Programming****, καθώς υπάρχουν τρεις ιεραρχικοί στόχοι.

Δεδομένα Πληθυσμού και Χωρητικότητας:

Περιοχή	Λευκοί (W)	Μαύροι (B)	Σύνολο	Χωρητικότητα Σχολείου
North	1,000	300	1,300	1,200
South	450	800	1,250	1,000
East	1,050	400	1,450	1,000
West	500	500	1,000	1,200
Σύνολο	3,000	2,000	5,000	4,400

Συνολική Αναλογία: 60% Λευκοί, 40% Μαύροι. **Υπερπληθυσμός:** Υπάρχουν 5,000 μαθητές για 4,400 θέσεις. Όλα τα σχολεία θα πρέπει να είναι υπερπλήρη κατά μέσο όρο $5000/4400 \approx 1.136$ (13.6%).

Μαθηματικό Μοντέλο

Μεταβλητές Απόφασης: x_{ijk} : Αριθμός μαθητών από την περιοχή i που πηγαίνουν στο σχολείο j της φυλής k . Όπου $i, j \in \{North, South, East, West\}$ και $k \in \{White, Black\}$.

Σκληροί Περιορισμοί (Hard Constraints): Όλοι οι μαθητές από κάθε περιοχή πρέπει να ανατεθούν σε κάποιο σχολείο.

$$\sum_j x_{ijk} = \text{Supply}_{ik} \quad \forall i, k$$

Περιορισμοί Στόχων (Goal Constraints):

- Priority 1: Φυλετική Ισορροπία (60% Λευκοί - 40% Μαύροι)** Για κάθε σχολείο j , ο αριθμός των Λευκών πρέπει να είναι το 60% του συνόλου.

$$W_j = 0.6(W_j + B_j) \Rightarrow 0.4W_j - 0.6B_j = 0 \Rightarrow 2W_j - 3B_j = 0$$

$$2 \sum_i x_{ij,White} - 3 \sum_i x_{ij,Black} + d_{1j}^- - d_{1j}^+ = 0$$

$$\text{Στόχος: } \min Z_1 = \sum_j (d_{1j}^- + d_{1j}^+)$$

- Priority 2: Ελαχιστοποίηση Μετακινήσεων (Max 30,000 μίλια)**

$$\sum_{i,j,k} x_{ijk} \cdot \text{Distance}_{ij} + d_2^- - d_2^+ = 30,000$$

$$\text{Στόχος: } \min Z_2 = d_2^+ \text{ (Ελαχιστοποίηση υπέρβασης).}$$

- Priority 3: Αναλογικός Υπερπληθυσμός** Ο στόχος πληθυσμού για κάθε σχολείο είναι: $\text{Target}_j = \text{Capacity}_j \times \frac{5000}{4400}$.

$$\sum_{i,k} x_{ijk} + d_{3j}^- - d_{3j}^+ = \text{Target}_j$$

$$\text{Στόχος: } \min Z_3 = \sum_j (d_{3j}^- + d_{3j}^+)$$

Αποτελέσματα Επίλυσης

Η επίλυση του μοντέλου έδειξε ότι **είναι εφικτό να ικανοποιηθούν πλήρως και οι τρεις στόχοι** (με απόκλιση 0).

- 1. Φυλετική Ισορροπία (P_1):** Επιτεύχθηκε απόλυτα. Σε κάθε σχολείο, η αναλογία είναι ακριβώς 60% Λευκοί και 40% Μαύροι.
- 2. Μετακινήσεις (P_2):** Επιτεύχθηκε. Το συνολικό κόστος μετακίνησης είναι **30,000 μίλια** (ακριβώς στο όριο).
- 3. Υπερπληθυσμός (P_3):** Επιτεύχθηκε. Ο μαθητικός πληθυσμός κατανεμήθηκε ακριβώς αναλογικά με τη χωρητικότητα κάθε σχολείου.

Πίνακας Ανάθεσης Μαθητών (Βέλτιστη Λύση):

Από Περιοχή	Προς Σχολείο	Λευκοί	Μαύροι	Σύνολο
North	North	578	300	878
	East	422	0	422
South	South	450	455	905
	East	0	345	345
East	North	240	245	485
	South	232	0	232
	East	260	109	369
	West	318	46	364
West	West	500	500	1,000

Table 4: Λεπτομερής κατανομή μαθητών για την επίτευξη όλων των στόχων.

Συγκεντρωτικά Στοιχεία ανά Σχολείο:

- **North High:** 1,364 μαθητές (818 W, 546 B). Στόχος: 1,364.
- **South High:** 1,136 μαθητές (682 W, 454 B). Στόχος: 1,136.
- **East High:** 1,136 μαθητές (682 W, 454 B). Στόχος: 1,136.
- **West High:** 1,364 μαθητές (818 W, 546 B). Στόχος: 1,364.

Συμπέρασμα: Η επιτροπή μπορεί να παρουσιάσει στο δικαστήριο ένα σχέδιο που επιτυγχάνει πλήρη φυλετική εξισορρόπηση, διατηρεί τα μίλια εντός προϋπολογισμού και μοιράζει δίκαια το βάρος του υπερπληθυσμού σε όλα τα σχολεία.

Άσκηση 6: Κατανομή Περιπολικών (Catawba Valley Highway Patrol)

Περιγραφή Προβλήματος

Ο Διοικητής Broderick Crawford πρέπει να κατανείμει 23 περιπολικά σε 6 οδικά τμήματα της περιοχής του. Κάθε τμήμα έχει διαφορετικά χαρακτηριστικά (κόστος λειτουργίας, μείωση ατυχημάτων, επαφές με οδηγούς, μείωση χρόνου απόκρισης). Το πρόβλημα διατυπώνεται ως ****Προγραμματισμός Στόχων με Προτεραιότητες (Preemptive Goal Programming)****, καθώς οι στόχοι έχουν αυστηρή ιεραρχία.

Δεδομένα ανά Οδικό Τμήμα:

Τμήμα	Κόστος (\$)	Μείωση Ατυχημάτων	Φυσικές Επαφές	Οπτικές Επαφές	Μείωση Χρόνου (min)
1	20	0.27	18	1700	0.32
2	18	0.21	26	900	0.65
3	22	0.28	10	650	0.43
4	24	0.19	34	230	0.87
5	17	0.23	25	1600	0.55
6	19	0.33	17	520	0.49

Μαθηματικό Μοντέλο

Μεταβλητές Απόφασης:

- x_i : Αριθμός περιπολικών που ανατίθενται στο τμήμα i ($i = 1, \dots, 6$).

Σκληροί Περιορισμοί (Hard Constraints):

- Σύνολο Περιπολικών: $\sum_{i=1}^6 x_i = 23$
- Όρια ανά Τμήμα: $2 \leq x_i \leq 5, \quad x_i \in \mathbb{Z}$

Περιορισμοί Στόχων (Goal Constraints) και Ιεραρχία:

- Προτεραιότητα P_1 - Κόστος Λειτουργίας (Max \$450):

$$\sum C_i x_i + d_1^- - d_1^+ = 450, \quad \min Z_1 = d_1^+$$

- Προτεραιότητα P_2 - Μείωση Ατυχημάτων (Min 5 ανά εκατ. μίλια):

$$\sum A_i x_i + d_2^- - d_2^+ = 5, \quad \min Z_2 = d_2^-$$

3. Προτεραιότητα P_3 - Φυσικές Επαφές (Min 350):

$$\sum P_i x_i + d_3^- - d_3^+ = 350, \quad \min Z_3 = d_3^-$$

4. Προτεραιότητα P_4 - Οπτικές Επαφές (Min 30,000):

$$\sum S_i x_i + d_4^- - d_4^+ = 30,000, \quad \min Z_4 = d_4^-$$

5. Προτεραιότητα P_5 - Χρόνος Απόκρισης (Max 15 min): Ο τρέχων χρόνος είναι 28 min. Για να φτάσουμε τα 15 min, απαιτείται μείωση κατά $28 - 15 = 13$ min.

$$\sum R_i x_i + d_5^- - d_5^+ = 13, \quad \min Z_5 = d_5^-$$

Αποτελέσματα και Ανάλυση

Η βέλτιστη κατανομή των 23 περιπολικών που ικανοποιεί ιεραρχικά τους στόχους είναι:

Οδικό Τμήμα	Αριθμός Περιπολικών (x_i)	Σχόλια
1	5	Μέγιστη ανάθεση (Υψηλές Οπτικές Επαφές)
2	5	Μέγιστη ανάθεση (Υψηλή μείωση χρόνου)
3	4	Ενδιάμεση ανάθεση
4	2	Ελάχιστη ανάθεση (Υψηλό κόστος)
5	5	Μέγιστη ανάθεση (Χαμηλό κόστος, Υψηλές επαφές)
6	2	Ελάχιστη ανάθεση
Σύνολο	23	

Επίτευξη Στόχων:

- Κόστος (P_1):** Επιτεύχθηκε πλήρως. Το συνολικό κόστος είναι **\$449** (Στόχος ≤ 450). Η υπέρβαση (d_1^+) είναι 0.
- Ατυχήματα (P_2):** Επιτεύχθηκε πλήρως. Η συνολική μείωση είναι **5.71** (Στόχος ≥ 5). Η υστέρηση (d_2^-) είναι 0.
- Φυσικές Επαφές (P_3):** Επιτεύχθηκε πλήρως. Οι συνολικές επαφές είναι **487** (Στόχος ≥ 350). Η υστέρηση (d_3^-) είναι 0.
- Οπτικές Επαφές (P_4):** **Δεν επιτεύχθηκε πλήρως.** Οι συνολικές επαφές είναι **25,100** (Στόχος 30,000). Υπάρχει υστέρηση 4,900 επαφών. Αυτό συμβαίνει διότι οι προηγούμενοι στόχοι (κυρίως το κόστος) περιόρισαν τις επιλογές, και τα τμήματα με υψηλές οπτικές επαφές (π.χ. Τμήμα 1 και 5) έχουν ήδη πάρει το μέγιστο αριθμό περιπολικών (5), αλλά δεν αρκούν για να πιάσουν το φιλόδοξο στόχο των 30,000.

5. **Χρόνος Απόκρισης (P_5): Δεν επιτεύχθηκε πλήρως.** Η συνολική μείωση χρόνου είναι 12.04 min, οδηγώντας σε μέσο χρόνο απόκρισης **15.96 min** (Στόχος 15 min). Η απόκλιση είναι 0.96 min.

Συμπέρασμα: Ο Διοικητής μπορεί να επιτύχει τους τρεις σημαντικότερους στόχους του (Κόστος, Ατυχήματα, Φυσικές Επαφές) πλήρως. Ωστόσο, οι πόροι (23 περιπολικά) και οι περιορισμοί κόστους δεν επαρκούν για να επιτευχθούν πλήρως οι στόχοι των Οπτικών Επαφών και του Χρόνου Απόκρισης ταυτόχρονα.

Άσκηση 7: Ασφάλεια Αεροδρομίων (Adeline Jonasson)

Περιγραφή Προβλήματος

Η Adeline Jonasson, επικεφαλής ομάδας εργασίας στην Transportation Security Administration (TSA), πρέπει να επιλέξει τον βέλτιστο συνδυασμό αναβαθμίσεων για δύο συστήματα ασφαλείας:

1. **Σύστημα Πύλης (Portal):** Ανιχνεύει όπλα/εκρηκτικά στους επιβάτες.
2. **Σύστημα Αποσκευών (Screening):** Ελέγχει τις χειραποσκευές.

Στόχος είναι η ελαχιστοποίηση των ψευδών συναγερμών (false alarms) τηρώντας παράλληλα συγκεκριμένους δημοσιονομικούς περιορισμούς.

a. Μεταβλητές Απόφασης

Ορίζουμε τις μεταβλητές απόφασης βάσει των "βημάτων" αναβάθμισης από τη βασική έκδοση.

- x_1 : Ο αριθμός των επιπλέον αναβαθμίσεων για το σύστημα Πύλης (Portal).
 - Κάθε μονάδα του x_1 μειώνει το ποσοστό ψευδών συναγερμών κατά 1%.
 - Εύρος: Από 10% (Basic) έως 2% (Max). Άρα $0 \leq x_1 \leq 8$.
- x_2 : Ο αριθμός των επιπλέον αναβαθμίσεων για το σύστημα Αποσκευών (Screening).
 - Κάθε μονάδα του x_2 μειώνει το ποσοστό ψευδών συναγερμών κατά 1%.
 - Εύρος: Από 6% (Basic) έως 3% (Max). Άρα $0 \leq x_2 \leq 3$.

b. Μαθηματικό Μοντέλο (Preemptive Goal Programming)

Παράμετροι Κόστους και Απόδοσης:

- **Portal (x_1):**

- Κόστος Αγοράς: $C_1 = 90,000 + 5,000x_1$
- Κόστος Συντήρησης: $M_1 = 15,000 + 1,500x_1$
- Ποσοστό Συναγερμών: $F_1 = 10 - 1x_1$ (%)

- **Screening (x_2):**

- Κόστος Αγοράς: $C_2 = 60,000 + 30,000x_2$
- Κόστος Συντήρησης: $M_2 = 9,000 + 1,200x_2$
- Ποσοστό Συναγερμών: $F_2 = 6 - 1x_2$ (%)

Ιεραρχία Στόχων:

1. **Priority 1:** Λειτουργικές απαιτήσεις (Ικανοποιείται από τα όρια $0 \leq x_1 \leq 8$ και $0 \leq x_2 \leq 3$).
2. **Priority 2:** Συνολικό ποσοστό ψευδών συναγερμών $\leq 10\%$ (0.1 ανά επιβάτη).

$$(10 - x_1) + (6 - x_2) + d_2^- - d_2^+ = 10$$

$$16 - (x_1 + x_2) + d_2^- - d_2^+ = 10 \Rightarrow x_1 + x_2 - d_2^- + d_2^+ = 6$$

Στόχος: $\min Z_2 = d_2^-$ (Ελαχιστοποίηση της υστέρησης από το 6, δηλαδή θέλουμε $x_1 + x_2 \geq 6$).

3. **Priority 3:** Συνολικό Κόστος Αγοράς $\leq \$250,000$.

$$(90,000 + 5,000x_1) + (60,000 + 30,000x_2) + d_3^- - d_3^+ = 250,000$$

$$150,000 + 5,000x_1 + 30,000x_2 + d_3^- - d_3^+ = 250,000$$

$$5,000x_1 + 30,000x_2 + d_3^- - d_3^+ = 100,000$$

Διαιρώντας με 5,000:

$$x_1 + 6x_2 + d_3'^- - d_3'^+ = 20$$

Στόχος: $\min Z_3 = d_3'^+$ (Ελαχιστοποίηση της υπέρβασης).

4. **Priority 4:** Συνολικό Κόστος Συντήρησης $\leq \$30,000$.

$$(15,000 + 1,500x_1) + (9,000 + 1,200x_2) + d_4^- - d_4^+ = 30,000$$

$$24,000 + 1,500x_1 + 1,200x_2 + d_4^- - d_4^+ = 30,000$$

$$1,500x_1 + 1,200x_2 + d_4^- - d_4^+ = 6,000$$

Διαιρώντας με 300:

$$5x_1 + 4x_2 + d_4'^- - d_4'^+ = 20$$

Στόχος: $\min Z_4 = d_4'^+$ (Ελαχιστοποίηση της υπέρβασης).

ε. Γραφική Επίλυση

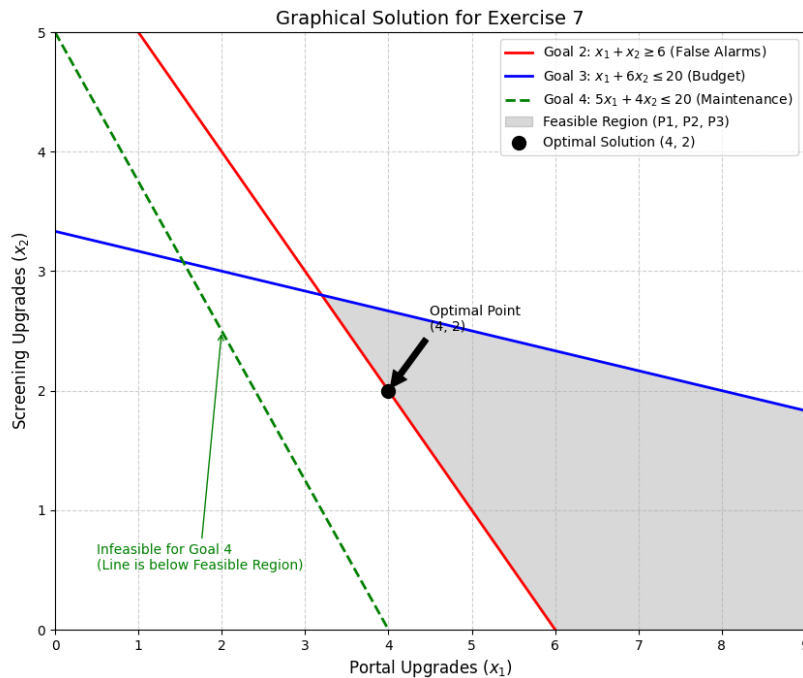


Figure 2: Γραφική Επίλυση Άσκησης 7: Η γκρι περιοχή δείχνει τις εφικτές λύσεις για τους Στόχους 2 και 3. Η πράσινη γραμμή (Στόχος 4) βρίσκεται εκτός αυτής της περιοχής, καθιστώντας τον ανέφικτο.

Στο διάγραμμα (x_1, x_2) με $0 \leq x_1 \leq 8$ και $0 \leq x_2 \leq 3$:

- Ο Στόχος 2 ($x_1 + x_2 \geq 6$) ορίζει την περιοχή πάνω από την ευθεία $x_1 + x_2 = 6$.
- Ο Στόχος 3 ($x_1 + 6x_2 \leq 20$) ορίζει την περιοχή κάτω από την ευθεία.
- Ο Στόχος 4 ($5x_1 + 4x_2 \leq 20$) ορίζει την περιοχή κάτω από την ευθεία.

Παρατηρούμε ότι για να ικανοποιηθεί ο Στόχος 2 (π.χ. σημείο $x_1 = 6, x_2 = 0$ ή $x_1 = 4, x_2 = 2$), είναι αδύνατο να ικανοποιηθεί ταυτόχρονα ο Στόχος 4. Για παράδειγμα, στο σημείο $(4, 2)$, η τιμή του περιορισμού 4 είναι $5(4) + 4(2) = 28 > 20$.

d. Επίλυση με Python (Preemptive)

Εφαρμόζοντας τη διαδοχική μέθοδο:

1. **Βήμα 1 (Priority 2 - False Alarms):** Το μοντέλο βρίσκει λύσεις που ικανοποιούν $x_1 + x_2 \geq 6$. Η απόκλιση είναι 0.
2. **Βήμα 2 (Priority 3 - Budget):** Το μοντέλο προσπαθεί να κρατήσει το κόστος κάτω από \$250,000 διατηρώντας $x_1 + x_2 \geq 6$. Βέλτιστη λύση σε αυτό το στάδιο: $x_1 = 4, x_2 = 2$. Κόστος: $150,000 + 5,000(4) + 30,000(2) = 230,000 \leq 250,000$. Η απόκλιση είναι 0.
3. **Βήμα 3 (Priority 4 - Maintenance):** Το μοντέλο προσπαθεί να ελαχιστοποιήσει το κόστος συντήρησης, κρατώντας σταθερά τα προηγούμενα επιτεύγματα. Με $x_1 = 4, x_2 = 2$, το κόστος συντήρησης είναι: $24,000 + 1,500(4) + 1,200(2) = 32,400$. Ο στόχος ήταν 30,000. Υπάρχει αναπόφευκτη υπέρβαση **\$2,400**.

Τελική Λύση:

- **Portal (x_1): 4 αναβαθμίσεις** (False Alarm: 6%, Κόστος: \$110,000, Maint: \$21,000)
- **Screening (x_2): 2 αναβαθμίσεις** (False Alarm: 4%, Κόστος: \$120,000, Maint: \$11,400)
- **Σύνολο:** False Alarm 10% (Επιτυχία), Κόστος \$230,000 (Επιτυχία), Maint \$32,400 (Αποτυχία κατά \$2,400).

e. Εναλλακτική Διατύπωση (Linear Programming)

Αν θέλουμε να ικανοποιήσουμε πλήρως όλους τους στόχους εκτός από τον τελευταίο (Maintenance), μπορούμε να διατυπώσουμε ένα πρόβλημα Γραμμικού Προγραμματισμού όπου οι Στόχοι 2 και 3 γίνονται σκληροί περιορισμοί και η αντικειμενική συνάρτηση ελαχιστοποιεί το κόστος συντήρησης.

Μοντέλο:

$$\min Z = 1500x_1 + 1200x_2$$

Subject to:

$$x_1 + x_2 \geq 6$$

$$x_1 + 6x_2 \leq 20$$

$$0 \leq x_1 \leq 8, \quad 0 \leq x_2 \leq 3$$

Ερμηνεία Ανεφικτότητας: Αν αυτό το μοντέλο δεν είχε εφικτή λύση, θα σήμαινε ότι οι στόχοι υψηλής προτεραιότητας (False Alarms και Budget) είναι ασύμβατοι μεταξύ τους. Δηλαδή, δεν θα υπήρχε κανένας συνδυασμός αναβαθμίσεων που να μειώνει τα ψευδή σήματα στο 10% και να κοστίζει κάτω από \$250,000. Στην περίπτωση μας, το μοντέλο έχει λύση, άρα η σύγκρουση εντοπίζεται μόνο στον τελευταίο στόχο (Συντήρηση).

2 Υλοποίηση Κώδικα Άσκηση 1

```
1 import pulp
2
3
4 prob = pulp . LpProblem ( " Aircraft_Fuel_Optimization " , pulp .
    LpMinimize )
5 airports = [      ATH      ,      LON      ,      PAR      ,      ROM      ]
6 prices = {      ATH      : 1.15 ,      LON      : 1.60 ,      PAR      : 1.70 ,
    ROM      : 1.65}
7 min_fuel = {      ATH      : 5000 ,      LON      : 5000 ,      PAR      : 5000
    ,      ROM      : 5000}
8 max_fuel = {      ATH      : 30000 ,      LON      : 30000 ,      PAR      :
    30000 ,      ROM      : 30000}
9 buy = pulp . LpVariable . dicts ( " Buy " , airports , lowBound =0)
10 depart = pulp . LpVariable . dicts ( " Depart " , airports , lowBound
    =0)
11 arrive = pulp . LpVariable . dicts ( " Arrive " , airports , lowBound
    =0)
12
13
14 prob += pulp . lpSum ([ prices [ index ] * buy [ index ] for index in
    airports ])
15
16 for index in airports :
17 prob += depart [ index ] ==arrive [ index ] + buy [ index ]
18
19
20 for index in airports :
21 prob += depart [ index ] >= min_fuel [ index ]
22 prob += depart [ index ] <= max_fuel [ index ]
23
24
```

```

25 prob . solve ()
26
27 print ( " Status : " , pulp . LpStatus [ prob . status ])
28 for index in airports :
29 print ( f " { index }: Buy { buy [ index ]. varValue } , Depart {
    depart [ index ]. varValue } " )
30 print ( f " Total Cost : { pulp . value ( prob . objective ) } " )

```

Listing 1: Σενάριο Βελτιστοποίησης με χρήση PuLP

3 Υλοποίηση Κώδικα Άσκηση 2

```

1 import pulp
2
3 vineyards = ['A', 'B', 'C']
4 factories = ['Crete', 'Ilia', 'Attica', 'Macedonia']
5 products = ['Bottled', 'Frozen', 'Jelly']
6
7 supply = {'A': 1400, 'B': 1100, 'C': 1700}
8 capacity = {'Crete': 1200, 'Ilia': 1100, 'Attica': 1400, 'Macedonia':
    1400}
9 demand = {'Bottled': 1200, 'Frozen': 900, 'Jelly': 700}
10
11 trans_cost = {
12     'A': {'Crete': 850, 'Ilia': 720, 'Attica': 910, 'Macedonia': 750},
13     'B': {'Crete': 970, 'Ilia': 790, 'Attica': 1050, 'Macedonia': 880},
14     'C': {'Crete': 900, 'Ilia': 830, 'Attica': 780, 'Macedonia': 820}
15 }
16
17 proc_cost = {
18     'Bottled': {'Crete': 2100, 'Ilia': 2350, 'Attica': 2200, 'Macedonia':
    1900},
19     'Frozen': {'Crete': 4100, 'Ilia': 4300, 'Attica': 3950, 'Macedonia':
    3900},
20     'Jelly': {'Crete': 2600, 'Ilia': 2300, 'Attica': 2500, 'Macedonia':
    2800}
21 }
22
23 rates = {'Bottled': 1.0, 'Frozen': 2.0, 'Jelly': 1.5}
24
25 prob = pulp.LpProblem("Wallasidis_Juice_Optimization", pulp.LpMinimize)
26
27 X = pulp.LpVariable.dicts("Transport", (vineyards, factories), lowBound
    =0)
28 Y = pulp.LpVariable.dicts("Production", (factories, products), lowBound
    =0)
29

```

```

30 prob += pulp.lpSum([trans_cost[idx][step] * X[idx][step] for idx in
    vineyards for step in factories]) + \
31     pulp.lpSum([proc_cost[tertiary][step] * Y[step][tertiary] for
    step in factories for tertiary in products])
32
33 for idx in vineyards:
34     prob += pulp.lpSum([X[idx][step] for step in factories]) <= supply[
    idx], f"Supply_{idx}"
35
36 for step in factories:
37     prob += pulp.lpSum([X[idx][step] for idx in vineyards]) <= capacity[
    step], f"Capacity_{step}"
38     prob += pulp.lpSum([X[idx][step] for idx in vineyards]) == pulp.
    lpSum([rates[tertiary] * Y[step][tertiary] for tertiary in products])
    , f"Balance_{step}"
39
40 for tertiary in products:
41     prob += pulp.lpSum([Y[step][tertiary] for step in factories]) ==
    demand[tertiary], f"Demand_{tertiary}"
42
43 prob.solve()
44
45 print(f"Total Cost: {pulp.value(prob.objective)}")
46 for idx in vineyards:
47     for step in factories:
48         if X[idx][step].varValue > 0: print(f"Trans {idx}->{step}: {X[
    idx][step].varValue}")
49 for step in factories:
50     for tertiary in products:
51         if Y[step][tertiary].varValue > 0: print(f"Prod {step}->{
    tertiary}: {Y[step][tertiary].varValue}")

```

Listing 2: Σενάριο Βελτιστοποίησης με χρήση PuLP

4 Υλοποίηση Κώδικα Άσκηση 3 C

```

1 import pulp
2
3 prob = pulp.LpProblem("Eva_Diet_Optimization", pulp.LpMaximize)
4
5 x1 = pulp.LpVariable("Meal_A", lowBound=0)
6 x2 = pulp.LpVariable("Meal_B", lowBound=0)
7
8 # Objective Function (Tastiness per portion)
9 # Meal A: 37g * 85 = 3145, Meal B: 65g * 95 = 6175
10 prob += 3145 * x1 + 6175 * x2
11

```

```

12 # Constraints
13 prob += 120 * x1 + 160 * x2 <= 450, "Calories"
14 prob += 5 * x1 + 10 * x2 <= 25, "Fat"
15 prob += 37 * x1 + 65 * x2 >= 120, "Min_Weight"
16
17 prob.solve()
18
19 print(f"Optimal Solution: Meal A = {x1.varValue}, Meal B = {x2.varValue}"
20       ")
21 print(f"Max Tastiness: {pulp.value(prob.objective)}")
22
23 print("\n--- Shadow Prices ---")
24 for name, c in prob.constraints.items():
25     print(f"{name}: {c.pi}")

```

Listing 3: Σενάριο Βελτιστοποίησης με χρήση PuLP

5 Υλοποίηση Κώδικα Άσκηση 3 Διαγράμματα

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 item = np.linspace(0, 5, 400)
5
6 y_calories = (450 - 120 * item) / 160
7
8 y_fat = (25 - 5 * item) / 10
9
10 y_weight = (120 - 37 * item) / 65
11
12 plt.figure(figsize = (10, 8))
13
14 plt.plot(item, y_calories, label = r"Calories ($120x_1 + 160x_2 \leq 450$)", color = "blue")
15 plt.plot(item, y_fat, label = r"Fat ($5x_1 + 10x_2 \leq 25$)", color = 'red')
16 plt.plot(item, y_weight, label = r"Weight ($37x_1 + 65x_2 \geq 120$)", color = "green")
17
18 y_upper = np.minimum(y_calories, y_fat)
19
20 y_lower = np.maximum(y_weight, 0)
21
22 plt.fill_between(item, y_lower, y_upper, where=(y_upper >= y_lower),
23                 color='gray', alpha=0.3, label="Feasible Region")
24
25 opt_x = 1.25

```

```

26 opt_y = 1.875
27 plt.plot(opt_x, opt_y, "ko", markersize=8, label='Optimal Solution
    (1.25, 1.875)')
28 plt.annotate(f"Optimal\total({opt_x}, {opt_y})", xy=(opt_x, opt_y),
29             xytext=(opt_x + 0.5, opt_y + 0.5),
30             arrowprops=dict(facecolor='black', shrink=0.05))
31
32 plt.xlim(0, 4)
33 plt.ylim(0, 3.5)
34 plt.xlabel("Meal A Portions ($x_1$)")
35 plt.ylabel("Meal B Portions ($x_2$)")
36 plt.title('Graphical Solution - Diet Optimization')
37 plt.legend()
38 plt.grid(True)
39
40 plt.show()

```

Listing 4: Σενάριο Βελτιστοποίησης με χρήση PuLP

6 Υλοποίηση Κώδικα Άσκηση 4

```

1 import pulp
2
3 def solve_scenario(name, weights, doctor_cost_per_unit, hard_budget=
    False, preemptive=False):
4     prob = pulp.LpProblem(name, pulp.LpMinimize)
5
6     x1 = pulp.LpVariable("Basic", lowBound=0, cat='Integer')
7     x2 = pulp.LpVariable("Advanced", lowBound=0, cat='Integer')
8     x3 = pulp.LpVariable("Supreme", lowBound=0, cat='Integer')
9
10    d1_minus = pulp.LpVariable("d1_under", lowBound=0)
11    d1_plus = pulp.LpVariable("d1_over", lowBound=0)
12    d2_minus = pulp.LpVariable("d2_under", lowBound=0)
13    d2_plus = pulp.LpVariable("d2_over", lowBound=0)
14    d3_minus = pulp.LpVariable("d3_under", lowBound=0)
15    d3_plus = pulp.LpVariable("d3_over", lowBound=0)
16
17    cost_sup = 720 + doctor_cost_per_unit
18
19    prob += x1 + x2 + x3 <= 40000
20    prob += 120*x1 + 180*x2 + 220*x3 <= 6000000
21
22    prob += 30*x1 + 35*x2 + 54*x3 + d1_minus - d1_plus == 2200000
23    prob += x3 + d2_minus - d2_plus == 3000
24    prob += 300*x1 + 350*x2 + cost_sup*x3 + d3_minus - d3_plus
    == 20000000

```

```

25
26     if hard_budget:
27         prob += d3_plus == 0
28
29     if preemptive:
30
31         prob += d3_plus == 0
32         prob += weights['w1'] * d1_minus + weights['w2'] * d2_minus
33     else:
34         prob += weights['w1'] * d1_minus + weights['w2'] * d2_minus +
weights['w3'] * d3_plus
35
36     prob.solve()
37     return x1.varValue, x2.varValue, x3.varValue, pulp.value(prob.
objective)
38
39 w_a = {'w1': 7/100000, 'w2': 1/1000, 'w3': 1/1000000}
40
41 w_b = {'w1': 10/55000, 'w2': 1/1000, 'w3': 1/1000000}
42
43 sol_a = solve_scenario("A", w_a, 330)
44 sol_b = solve_scenario("B", w_b, 330)
45 sol_c = solve_scenario("C", w_b, 440)
46 sol_d = solve_scenario("D", w_a, 330, hard_budget=True)
47 sol_e = solve_scenario("E", w_a, 330, preemptive=True)
48
49 print(f"Scenario A: {sol_a}")
50 print(f"Scenario B: {sol_b}")
51 print(f"Scenario C: {sol_c}")
52 print(f"Scenario D: {sol_d}")
53 print(f"Scenario E: {sol_e}")

```

Listing 5: Σενάριο Βελτιστοποίησης με χρήση PuLP

7 Υλοποίηση Κώδικα Άσκηση 5

```

1 import pulp
2
3 def solve_school_busing():
4     # ---
5     districts = ['North', 'South', 'East', 'West']
6     schools = ['North', 'South', 'East', 'West']
7     races = ['White', 'Black']
8
9     #
10
11     supply = {

```

```

11     'North': {'White': 1000, 'Black': 300},
12     'South': {'White': 450, 'Black': 800},
13     'East': {'White': 1050, 'Black': 400},
14     'West': {'White': 500, 'Black': 500}
15 }
16
17 #
18 capacities = {'North': 1200, 'South': 1000, 'East': 1000, 'West':
19 1200}
20
21 #
22
23 total_students = 5000
24 total_capacity = 4400
25 prop_factor = total_students / total_capacity # ~1.136
26 targets_pop = {s: capacities[s] * prop_factor for s in schools}
27
28 #
29
30 distances = {
31     'North': {'North': 0, 'South': 30, 'East': 12, 'West': 20},
32     'South': {'North': 30, 'South': 0, 'East': 18, 'West': 26},
33     'East': {'North': 12, 'South': 18, 'East': 0, 'West': 24},
34     'West': {'North': 20, 'South': 26, 'East': 24, 'West': 0}
35 }
36
37 target_miles = 30000
38
39 # ---
40
41 prob = pulp.LpProblem("School_Busing", pulp.LpMinimize)
42
43 #
44 : x[district][school][race]
45
46
47 x = pulp.LpVariable.dicts("Assign", (districts, schools, races),
48 lowBound=0)
49
50 #
51 (Deviational Variables)
52 d1_m = pulp.LpVariable.dicts("d1_minus", schools, lowBound=0)
53 d1_p = pulp.LpVariable.dicts("d1_plus", schools, lowBound=0)
54 d2_m = pulp.LpVariable("d2_minus", lowBound=0)
55 d2_p = pulp.LpVariable("d2_plus", lowBound=0)
56 d3_m = pulp.LpVariable.dicts("d3_minus", schools, lowBound=0)
57 d3_p = pulp.LpVariable.dicts("d3_plus", schools, lowBound=0)
58
59 #
60 :
61
62 for d in districts:

```

```

53         for r in races:
54             prob += pulp.lpSum([x[d][s][r] for s in schools]) == supply[
d][r]
55
56         # 1: (60% White -
40% Black)
57         # : 2*White - 3*Black = 0
58         for s in schools:
59             white_in_s = pulp.lpSum([x[d][s]['White'] for d in districts])
60             black_in_s = pulp.lpSum([x[d][s]['Black'] for d in districts])
61             prob += 2 * white_in_s - 3 * black_in_s + d1_m[s] - d1_p[s] == 0
62
63         # 2: <= 30,000
64         total_miles = pulp.lpSum([x[d][s][r] * distances[d][s]
65                                     for d in districts for s in schools for r
in races])
66         prob += total_miles + d2_m - d2_p == target_miles
67
68         # 3:
69         for s in schools:
70             total_in_s = pulp.lpSum([x[d][s][r] for d in districts for r in
races])
71             prob += total_in_s + d3_m[s] - d3_p[s] == targets_pop[s]
72
73         # --- (Preemptive Goal Programming) ---
74
75         # Priority 1:
76         prob.setObjective(pulp.lpSum([d1_m[s] + d1_p[s] for s in schools]))
77         prob.solve()
78         # P1
79         val_p1 = pulp.value(prob.objective)
80         prob += pulp.lpSum([d1_m[s] + d1_p[s] for s in schools]) == val_p1
81
82         # Priority 2:
83         prob.setObjective(d2_p)
84         prob.solve()
85         # P2
86         val_p2 = pulp.value(prob.objective)
87         prob += d2_p == val_p2
88
89         # Priority 3:
90         prob.setObjective(pulp.lpSum([d3_m[s] + d3_p[s] for s in schools]))
91         prob.solve()
92

```

```

93     return x, pulp.value(total_miles)
94
95 #
96 assignments, final_miles = solve_school_busing()
97 print(f"Total Miles: {final_miles}")

```

Listing 6: Σενάριο Βελτιστοποίησης με χρήση PuLP

8 Υλοποίηση Κώδικα Άσκηση 6

```

1 import pulp
2
3 segments = [1, 2, 3, 4, 5, 6]
4 costs = {1: 20, 2: 18, 3: 22, 4: 24, 5: 17, 6: 19}
5 accidents = {1: 0.27, 2: 0.21, 3: 0.28, 4: 0.19, 5: 0.23, 6: 0.33}
6 physical = {1: 18, 2: 26, 3: 10, 4: 34, 5: 25, 6: 17}
7 sight = {1: 1700, 2: 900, 3: 650, 4: 230, 5: 1600, 6: 520}
8 response = {1: 0.32, 2: 0.65, 3: 0.43, 4: 0.87, 5: 0.55, 6: 0.49}
9
10 targets = {"cost": 450, 'acc': 5, "phys": 350, "sight": 30000, 'resp':
11            13}
12
13 prob = pulp.LpProblem('Patrol_Allocation', pulp.LpMinimize)
14
15 item = pulp.LpVariable.dicts("Cars", segments, lowBound= 2, upBound= 5,
16                               cat= "Integer")
17
18 d1_m = pulp.LpVariable(
19     "d1_m", lowBound= 0); d1_p = pulp.LpVariable("d1_p", lowBound= 0
20 )
21
22 d2_m = pulp.LpVariable('d2_m', lowBound= 0); d2_p = pulp.LpVariable("
23     d2_p", lowBound=0)
24
25 d3_m = pulp.LpVariable('d3_m', lowBound=0); d3_p = pulp.LpVariable("d3_p
26     ", lowBound=0)
27
28 d4_m = pulp.LpVariable(
29     'd4_m', lowBound=0); d4_p = pulp.LpVariable('d4_p', lowBound=0
30 )
31
32 d5_m = pulp.LpVariable(
33     "d5_m", lowBound=0); d5_p = pulp.LpVariable('d5_p', lowBound=0
34 )
35
36 prob += pulp.lpSum([item[iter] for iter in segments]) == 23
37
38 prob += pulp.lpSum([costs[iter] * item[iter] for iter in segments]) +
39     d1_m - d1_p == targets['cost']
40
41 prob += pulp.lpSum([accidents[iter]*item[iter] for iter in segments]) +
42     d2_m - d2_p == targets['acc']
43
44 prob += pulp.lpSum([physical[iter]*item[iter] for iter in segments]) +
45     d3_m - d3_p == targets["phys"]

```

```

31 prob += pulp.lpSum([sight[iter]*item[iter] for iter in segments]) + d4_m
    - d4_p == targets['sight']
32 prob += pulp.lpSum([response[iter]*item[iter] for iter in segments]) +
    d5_m - d5_p == targets['resp']
33
34 prob.setObjective(d1_p)
35 prob.solve()
36 prob += d1_p == d1_p.varValue
37
38 prob.setObjective(d2_m)
39 prob.solve()
40 prob += d2_m == d2_m.varValue
41
42 prob.setObjective(d3_m)
43 prob.solve()
44 prob += d3_m == d3_m.varValue
45
46 prob.setObjective(d4_m)
47 prob.solve()
48 prob += d4_m == d4_m.varValue
49
50 prob.setObjective(d5_m)
51 prob.solve()
52
53 print('Assignments:', {iter: item[iter].varValue for iter in segments})
54 print(f'Final Cost: {combined(costs[iter]*item[iter].varValue for iter
    in segments)}')
55 print(f'Final Response Time: {28 - combined(response[iter]*item[iter].
    varValue for iter in segments)}')

```

Listing 7: Σενάριο Βελτιστοποίησης με χρήση PuLP

9 Υλοποίηση Κώδικα Άσκηση 7

```

1 import pulp
2
3 def solve_exercise_7():
4
5     c1_base, m1_base, c1_step, m1_step = 90000, 15000, 5000, 1500
6
7     c2_base, m2_base, c2_step, m2_step = 60000, 9000, 30000, 1200
8
9     target_fa_total = 10
10    target_budget = 250000
11    target_maint = 30000
12
13    prob = pulp.LpProblem("Airport_Security", pulp.LpMinimize)

```

```

14
15     x1 = pulp.LpVariable("Portal_Upgrade", lowBound=0, upBound=8, cat='
Integer')
16     x2 = pulp.LpVariable("Screening_Upgrade", lowBound=0, upBound=3, cat
='Integer')
17
18     d2_m = pulp.LpVariable("d2_minus", lowBound=0); d2_p = pulp.
LpVariable("d2_plus", lowBound=0)
19     d3_m = pulp.LpVariable("d3_minus", lowBound=0); d3_p = pulp.
LpVariable("d3_plus", lowBound=0)
20     d4_m = pulp.LpVariable("d4_minus", lowBound=0); d4_p = pulp.
LpVariable("d4_plus", lowBound=0)
21
22     current_fa = (10 - 1*x1) + (6 - 1*x2)
23     current_cost = (c1_base + c1_step*x1) + (c2_base + c2_step*x2)
24     current_maint = (m1_base + m1_step*x1) + (m2_base + m2_step*x2)
25
26     prob += current_fa + d2_m - d2_p == target_fa_total
27
28     prob += current_cost + d3_m - d3_p == target_budget
29
30     prob += current_maint + d4_m - d4_p == target_maint
31
32     prob.setObjective(d2_p)
33     prob.solve()
34     prob += d2_p == d2_p.varValue
35
36     prob.setObjective(d3_p)
37     prob.solve()
38     prob += d3_p == d3_p.varValue
39
40     prob.setObjective(d4_p)
41     prob.solve()
42
43     return x1.varValue, x2.varValue, pulp.value(prob.objective)
44
45 x1_sol, x2_sol, maint_overage = solve_exercise_7()
46 print(f"Optimal Upgrades: Portal={x1_sol}, Screening={x2_sol}")
47 print(f"Maintenance Overage: {maint_overage}")

```

Listing 8: Σενάριο Βελτιστοποίησης με χρήση PuLP